

BUREAU D'ETUDE ROBOT MOBILE

SUJET DE TD & TP 4AE-SE

**DEPARTEMENT DE GENIE ÉLEC-
TRIQUE ET INFORMATIQUE**

**Responsable pédagogique : Claude BA-
RON
Support : Sébastien DI MERCURIO**

Travail demandé.....	4
Ressources.....	4
Lexique	4
1. Présentation de la plateforme de TP	5
2. Présentation du projet	7
2.1 Diagramme de contexte	7
2.2 Cahier des charges du superviseur.....	7
3.2.1 Communication entre le superviseur et le moniteur	7
3.2.2 Communication entre le superviseur et le robot	8
3.2.3 Démarrage robot	9
3.2.4 Déplacement et état du robot	9
3.2.4 Gestion de la caméra.....	9
3. Travail à faire	10
3.1 Spécification du superviseur.....	10
3.2 Conception du superviseur.....	12
4. Travail de TP.....	14
4.1 Implantation (codage et tests) du superviseur.....	14

TRAVAIL DEMANDE

1. Le travail sera réalisé en groupes de deux (ou trois si cas particulier) étudiants sur 2 séances de TD et 4 séances de TP (plus 2 séances de TD pour les FISA).
2. Le projet sera évalué en contrôle continu. Il donnera lieu à un rapport et une démonstration (la démonstration se fera en anglais pour les étudiants de FISA, auxquels il sera également demandé un court résumé écrit en anglais, à joindre au rapport). Une trame pour le rapport est proposée sur la page Moodle de l'enseignement.
3. L'évaluation a pour but de juger de vos compétences pour rédiger un compte rendu, réaliser la conception d'une application temps réel et programmer sur un système temps réel, mais aussi sur vos connaissances plus générales des systèmes temps réel (les étudiants en FISA seront également évalués par ailleurs sur l'anglais, voir critères avec l'enseignant).
4. Le code réalisé sera rendu sous forme d'archive comprenant les codes sources modifiés et commentés.
5. Le rendu du rapport et du code (zippé) se fera sous la forme d'une archive à envoyer à votre encadrant de TP (et à déposer sous Moodle).
6. La démonstration (livraison au client de l'application = votre encadrant de TP) aura lieu durant la dernière séance de TP.

RESSOURCES

1. Page Moodle : <https://moodle.insa-toulouse.fr/course/view.php?id=235>
2. Dépôt GEI-INSA sur GitHub : <https://github.com/INSA-GEI/dumber.git>

LEXIQUE

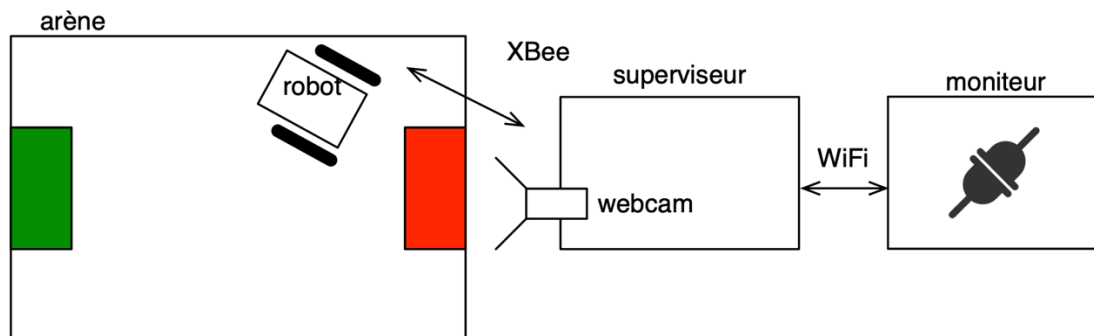
- Dépôt (git) : En informatique, un dépôt correspond à l'ensemble des documents informatiques d'un projet gérés en gestion de version. Cela se traduit par un archivage en réseau qui, une fois téléchargé sur l'ordinateur, correspond au contenu d'un répertoire donné.
- Git : Git est un gestionnaire de version couramment utilisé en informatique. Son rôle est de gérer les évolutions faites dans les fichiers d'un dépôt et assurer la synchronisation avec archivage réseau, dans notre cas Github.
- Netbeans : c'est un environnement de développement (ou IDE, pour Integrated Development Environment) qui vous servira à écrire, compiler et télécharger le programme de supervision du robot dans la carte Raspberry Pi.
- Terminal : Une fenêtre (généralement noire), où l'on peut taper des commandes exécutées par le système Ubuntu sur le PC de la salle de TP.

1. PRESENTATION DE LA PLATEFORME DE TP

Dans ce projet, votre rôle sera de concevoir, coder et mettre au point le programme de contrôle et de supervision (appelé ci-après *superviseur*) d'un robot mobile piloté à distance.

Le projet utilise une plateforme physique constituée de (voir Figure 1) :

- Une arène : terrain dans lequel le robot évolue.
- Un robot mobile : robot deux roues embarquant un microcontrôleur, une puce Xbee assurant la liaison avec le superviseur, et un ensemble de composants matériels nécessaires à son déplacement. Un symbole distinctif sur le dos permet de l'identifier et de le localiser dans l'arène. L'intelligence embarquée est volontairement limitée au contrôle de son déplacement (contrôle des moteurs) et à des fonctionnalités pour connaître son état. (Notez que l'application que vous allez développer est le superviseur, pas le code embarqué sur le robot)
- Un ordinateur Raspberry Pi 3 : ordinateur minimaliste et embarquable, équivalent en puissance d'un smartphone d'entrée de gamme, sur laquelle est installé Xenomai. Un module Xbee a été ajouté sur l'ordinateur pour communiquer avec le robot. Sur cet ordinateur s'exécutera une entité logicielle appelée *superviseur* chargée du contrôle et de la supervision du robot ; elle orchestrera le fonctionnement de la plateforme en assurant le respect des contraintes temporelles du système. Le superviseur communique avec le moniteur par un socket ; le serveur est mis en place sur le superviseur, le moniteur en est le client, la liaison physique est assurée par WiFi. L'ordinateur est couplé (par liaison USB) à une webcam fixée au-dessus de l'arène qui permet de faire une acquisition visuelle de l'arène et de localiser le robot.
- Un poste de travail informatique (celui de la salle de TP) aura un rôle double : 1) en support au programmeur : pour mettre en place l'environnement logiciel du TP, coder, compiler et exécuter le programme de supervision (interface de développement), et 2) en support à l'utilisateur du robot : pour contrôler (envoi de consignes) et superviser à distance l'état (position, charge de la batterie...) et les déplacements du robot via un entité logicielle appelée *moniteur* avec laquelle l'opérateur du robot interagit via une interface graphique (voir Figure 2). Le moniteur sert pour le contrôle du robot et la visualisation d'une scène. Son code vous sera fourni.



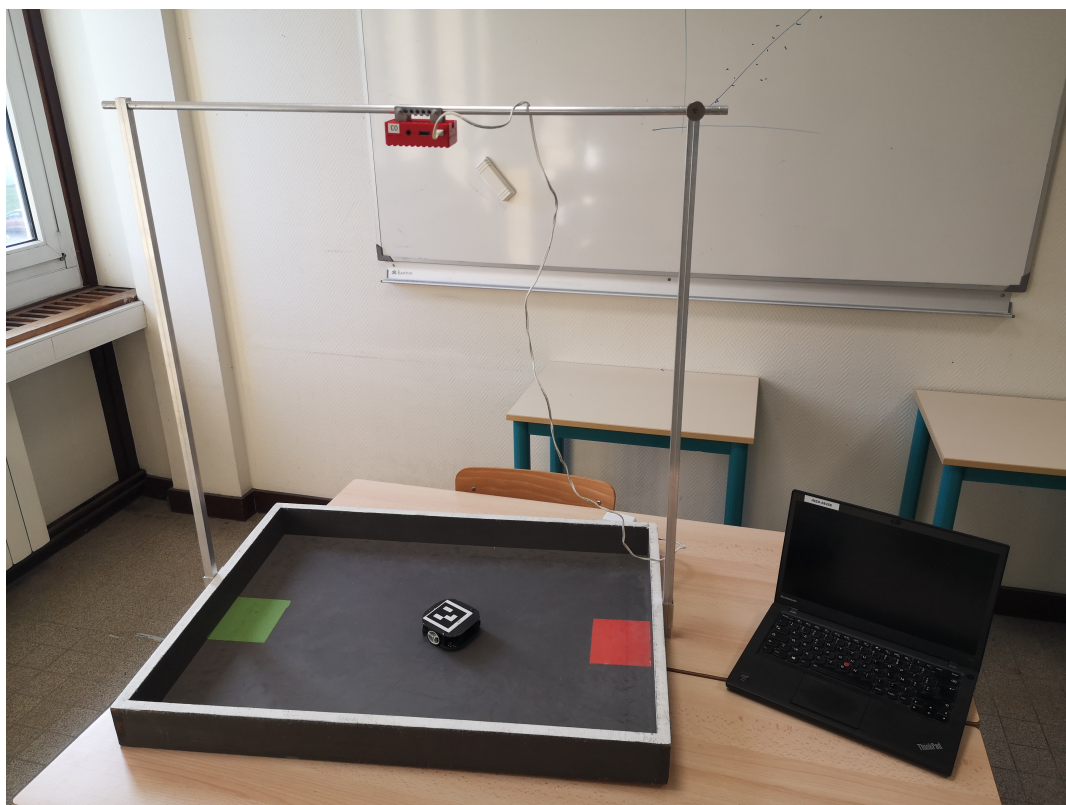


Figure 1. Vue schématique et photo de la plateforme

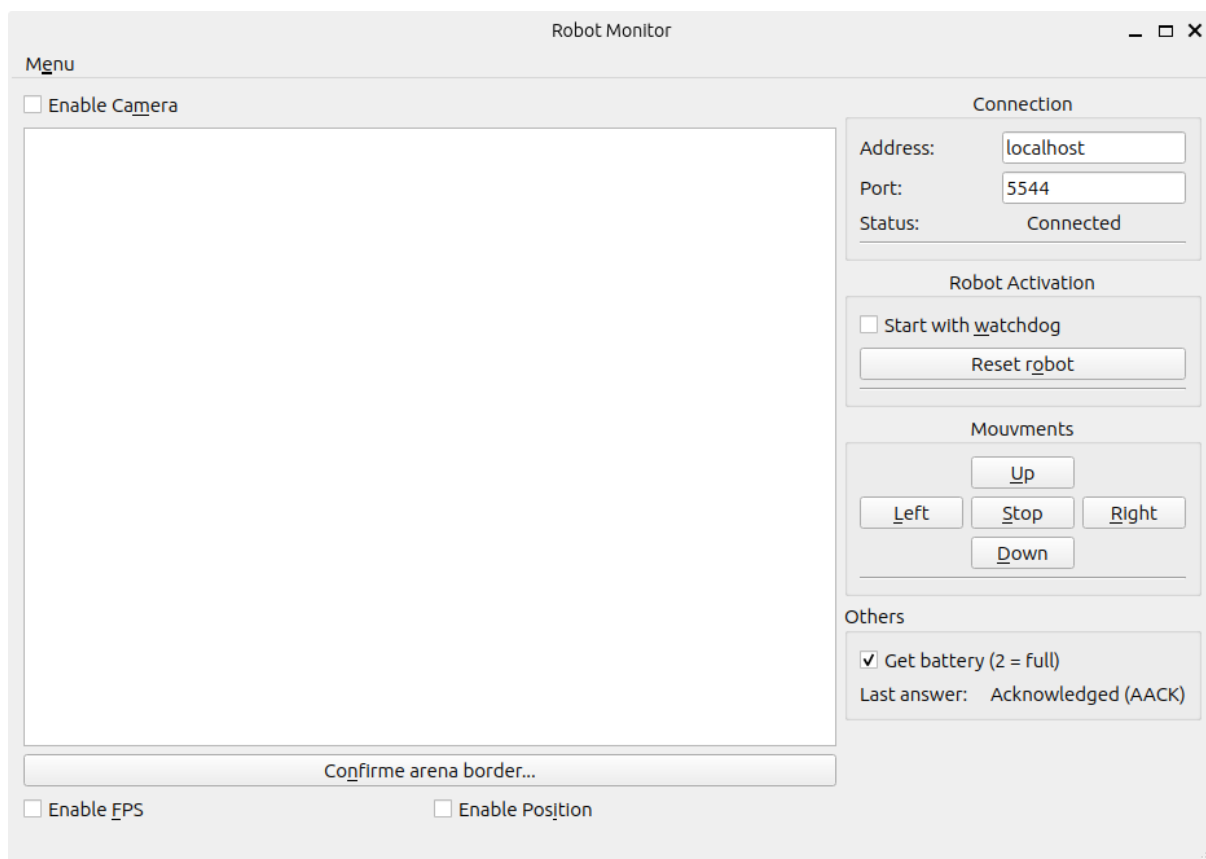


Figure 2. Interface graphique du moniteur en cours d'exécution

Les différents paramètres sont initialement grisés, jusqu'à ce que le moniteur soit connecté au superviseur et reçoive les premières données en provenance du superviseur.

2. PRESENTATION DU PROJET

L'objectif est de concevoir et de développer l'architecture logicielle du superviseur afin d'assurer le fonctionnement de la plateforme. Les missions du superviseur sont :

1. Assurer la communication entre le moniteur et le superviseur
2. Assurer la communication entre le superviseur et le robot
3. Superviser l'état du robot
4. Contrôler le déplacement du robot
5. Contrôler et diffuser les images de la webcam

2.1 Diagramme de contexte

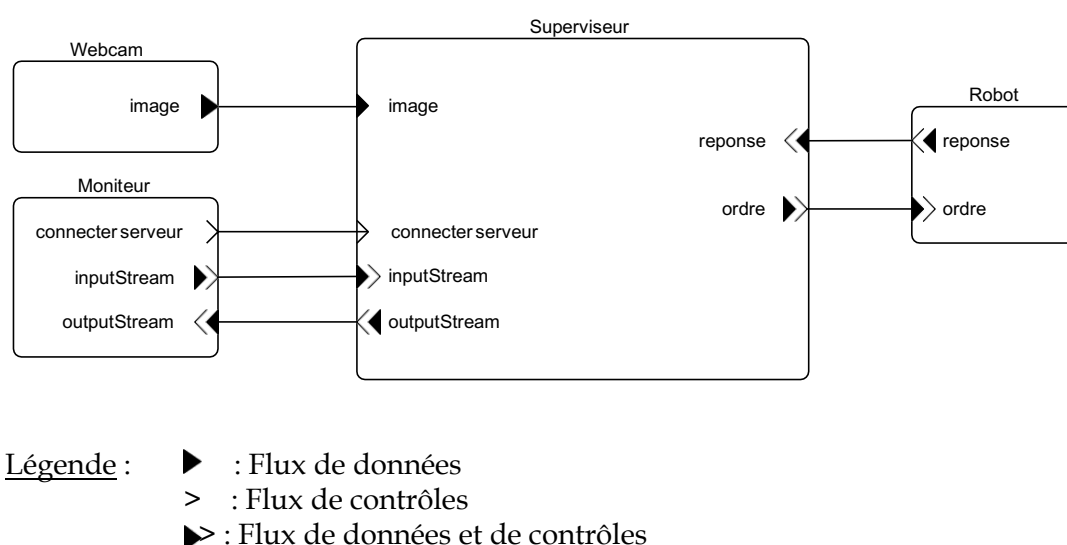


Figure 3. Diagramme de contexte du superviseur

La webcam produit des images (données) sous la forme d'un tableau d'octets. Le robot reçoit des ordres (contrôles) sous la forme d'une chaîne de caractères et retourne une réponse aussi sous la forme d'une chaîne. Le moniteur envoie un événement (connecter serveur) pour demander la connexion avec le serveur puis établit une communication bi-directionnelle sous la forme d'un flux d'octets (inputStream et outputStream) ; le moniteur peut envoyer des requêtes et le superviseur peut envoyer des données pour mettre à jour certains éléments de l'interface. Les messages sont prédéfinis et sont présentés dans l'annexe A.2.

2.2 Cahier des charges du superviseur

3.2.1 Communication entre le superviseur et le moniteur

Lancement du serveur.

Exigence 1 : Le lancement du serveur doit être réalisé au démarrage du superviseur. En cas d'échec du démarrage du serveur, un message textuel doit être affiché sur la console de lancement de l'application. Ce message doit signaler le problème et le superviseur doit s'arrêter.

Établissement du socket. La connexion entre le moniteur et le superviseur est réalisée suite à la demande de l'utilisateur via l'interface graphique. Lorsque la demande est faite, un socket est créé, il faut donc que le serveur soit en attente d'une demande de connexion.

Exigence 2 : La connexion entre le moniteur et le superviseur (via le socket) doit être établie suite à la demande de connexion de l'utilisateur.

Réception des messages.

Exigence 3 : Tous les messages envoyés depuis le moniteur doivent être réceptionnés par le superviseur.

Envoi des messages.

Exigence 4 : Le superviseur doit traiter les messages à envoyer au moniteur en moins de 10 ms.

Détection de la perte de communication. La perte de communication entre le moniteur et le superviseur est détectée lors de la lecture sur le socket, qui retourne un message d'erreur si la communication est perdue.

Exigence 5 : Le superviseur doit détecter la perte de communication avec le moniteur. En cas de perte de la communication un message doit être affiché sur la console de lancement du superviseur.

Reprise de la communication. La communication entre le moniteur et le superviseur est fermée par le superviseur.

Exigence 6 : En cas de perte de communication entre le superviseur et moniteur, le superviseur doit stopper le robot, la communication avec le robot, fermer le serveur et déconnecter la caméra afin de revenir dans le même état qu'au démarrage du superviseur.

3.2.2 Communication entre le superviseur et le robot

La communication avec le robot s'effectue par le biais d'émetteur-récepteur Xbee. Pour communiquer, il est nécessaire de commencer par ouvrir la communication entre le superviseur et le boîtier Xbee.

Mettre en place la communication avec le robot. L'ouverture de la communication (c'est-à-dire la réservation d'un port série) avec le robot est automatiquement demandée par le moniteur dès que le socket est en place.

Exigence 7 : Dès que la communication avec le moniteur est en place, la communication entre le superviseur et le robot doit être ouverte. Si la communication est active, il faut envoyer un message d'acquiescement au moniteur. En cas d'échec, un message le signalant est renvoyé au moniteur.

Surveillance de la communication avec le robot. La communication entre le robot et le superviseur peut être perdue. Afin de surveiller cela, il faut mettre en place un mécanisme permettant d'inférer cette perte. En cas d'échec de communication, la bibliothèque de fonction fournie retourne un message contenant un code d'erreur.

Perte de la communication avec le robot.

Exigence 9 : Lorsque la communication entre le robot et le superviseur est perdue, le superviseur doit envoyer un message spécifique au moniteur. Le superviseur doit alors fermer la communication entre le robot et le superviseur et se remettre dans un état initial permettant de relancer la communication.

3.2.3 Démarrage robot

Démarrage du robot. Le robot a deux modes de démarrage. Un mode simple, dit 'sans watchdog' et un mode évolué qui fera l'objet d'une question ultérieure.

Exigence 10 : Lorsque l'utilisateur demande, via le moniteur, le démarrage sans watchdog, le superviseur doit démarrer le robot dans ce mode. En cas de succès, un message d'acquiescement est retourné au moniteur. En cas d'échec, un message indiquant l'échec est transmis au moniteur.

3.2.4 Déplacement et état du robot

Le robot n'a aucune intelligence et ne réagit qu'aux ordres qu'il reçoit.

Déplacement manuel du robot. Le robot peut recevoir cinq ordres de mouvement (avancer, reculer, tourner à droite, tourner à gauche et stopper). Lorsque l'utilisateur presse les flèches de l'interface graphique un message est envoyé au superviseur avec le mouvement à réaliser.

Exigence 11 : Lorsque qu'un ordre de mouvement est reçu par le superviseur, Le superviseur doit transmettre la commande au robot en moins de 100 ms.

Niveau de batterie du robot. Il est possible de récupérer le niveau de la batterie du robot. La valeur retournée est ensuite à transmettre au moniteur pour permettre à l'utilisateur de surveiller le niveau de charge du robot

Exigence 12 : Le superviseur doit mettre à jour le niveau de la batterie du robot sur le moniteur toutes les 500 ms.

3.2.4 Gestion de la caméra

Ouverture de la caméra.

Exigence 13 : Le superviseur doit démarrer la caméra suite à une demande provenant du moniteur. Si l'ouverture de la caméra a échoué, le superviseur doit envoyer un message au moni-

Capture d'une image (mode nominal). L'envoi de l'image au moniteur s'effectue par l'envoi d'un message (l'image est compressée lors de l'envoi du message). La fréquence de capture d'une image est paramétrable.

Exigence 14 : Dès que la caméra est ouverte, le superviseur doit envoyer une image au moniteur toutes les 100 ms.

Fermeture de la caméra.

Exigence 15 : Le superviseur doit fermer la caméra suite à une demande provenant du moniteur. Il envoie alors un message au moniteur pour signifier l'acquittement de la demande et stopper l'envoi périodique des images.

Calibration de l'arène. Pour accélérer le traitement de l'image et déterminer la position du robot, il faut limiter la zone d'étude à l'arène. Pour cela il est nécessaire de faire un pré-traitement qui recherche l'arène.

Exigence 16 : Suite à une demande de recherche de l'arène, le superviseur doit stopper l'envoi périodique des images, faire la recherche de l'arène et renvoyer une image sur laquelle est dessinée cette arène. Si aucune arène n'est trouvée, le superviseur doit envoyer un message d'échec au moniteur.

L'utilisateur doit ensuite valider visuellement via le moniteur si l'arène a bien été trouvée. L'utilisateur peut :

- valider l'arène : dans ce cas, le superviseur doit sauvegarder l'arène trouvée (pour l'utiliser ultérieurement) puis retourner dans son mode d'envoi périodique des images en ajoutant à l'image l'arène dessinée.
- annuler la recherche : dans ce cas, le superviseur doit simplement retourner dans son mode d'envoi périodique des images et invalider la recherche.

Calcul de la position du robot. Se fait par traitement d'une image pour trouver la position du robot. Il est possible de dessiner sur l'image la position trouvée. La position est envoyée du superviseur vers le moniteur en utilisant un message spécifique.

Exigence 17 : Suite à une demande de l'utilisateur de calculer la position du robot, le superviseur doit calculer cette position, dessiner sur l'image le résultat et envoyer un message au moniteur avec la position toutes les 100 ms. Si le robot n'a pas été trouvé, le superviseur doit envoyer un message de position avec la valeur (-1,-1).

Stopper le calcul de la position du robot. Il est possible pour l'utilisateur de demander l'arrêt du calcul de la position.

Exigence 18 : Suite à une demande de l'utilisateur de stopper le calcul de la position du robot, le superviseur doit rebasculer dans un mode d'envoi de l'image sans le calcul de la position.

3. TRAVAIL A FAIRE

Durant le TD, vous allez dérouler une démarche d'ingénierie en suivant les différentes étapes.

3.1 Spécification du superviseur

L'étape de spécification permet de déterminer, à partir des missions et des exigences du superviseur, les fonctions que le superviseur devra assurer (pour rappel, une fonction transforme un flux de données d'entrée en flux de sortie, en consommant du temps et des ressources).

L'objectif est ici d'obtenir un **diagramme d'architecture fonctionnelle** montrant les échanges entre les fonctions (caractérisation des fonctions et de leurs interfaces, flux de données par les échanges d'entrées et sorties, flux de contrôles (activations, synchronisations), point de vue temporel).

Plusieurs diagrammes, complémentaires, sont nécessaires pour décrire les fonctions, leurs interfaces et leur comportement ; pour cela, on adopte d'une part un point de vue statique, pour décrire les échanges de données et de contrôles entre les fonctions, indépendamment des instants auxquels se produisent ces échanges, et d'autre part, un point de vue dynamique, pour décrire les échanges dans le temps. Le comportement de chaque fonction est fourni dans le document « Annexes techniques ».

A cette étape, vous devez donc établir seulement le diagramme d'architecture fonctionnelle statique.

Démarche à suivre : Sur la base des missions du superviseur, traduire et raffiner une à une les exigences en fonctions. Vous caractérisez ces fonctions par leur nom (sous forme de verbe à l'infinitif), leurs entrées (données consommées et contrôles), leurs sorties (données et contrôles produits), leur comportement (traitements réalisés). L'interconnexion de ces fonctions les unes avec les autres vous donne progressivement un schéma d'architecture fonctionnelle statique.

Vous pouvez, pour le représenter, utiliser le formalisme des Figures 4 à 6, inspiré de SA-RT, une méthode d'analyse fonctionnelle adaptée au temps réel.

La Figure 4 vous aide pour initier la démarche. A partir de l'analyse de l'exigence 1, vous définissez la fonction 'Démarrer serveur' que vous représentez dans un rectangle blanc. Une note écrite sur fond jaune décrit en substance le comportement de la fonction. La flèche en pointillés qui arrive en entrée de la fonction représente un événement.

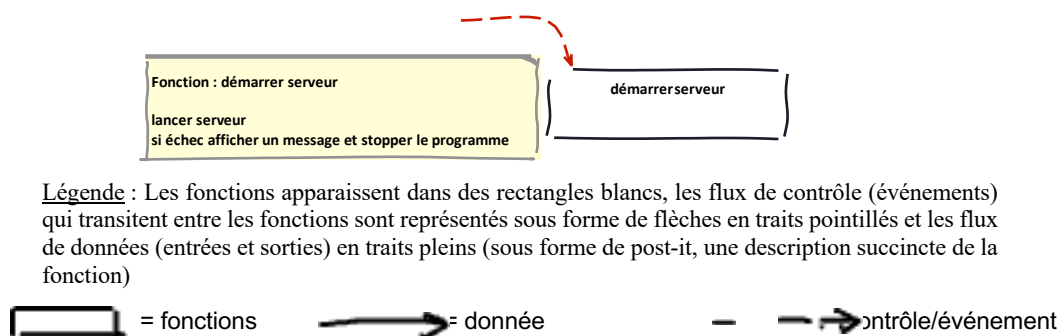


Figure 4. Initiation de la construction du diagramme d'architecture fonctionnelle statique du superviseur

Et ainsi de suite... Vous complétez le diagramme en considérant l'exigence 2 (voir Figure 5). Vous créez la fonction 'Établir connexion avec moniteur'. Vous modifiez aussi la fonction 'Démarrer serveur' pour que celle-ci produise l'événement 'Serveur démarré'. L'événement 'Ouvrir socket' est produit par le moniteur lorsque l'utilisateur demande la connexion entre le moniteur et le superviseur. Le résultat de l'intégration de cette nouvelle fonction est donné sur la Figure 5.

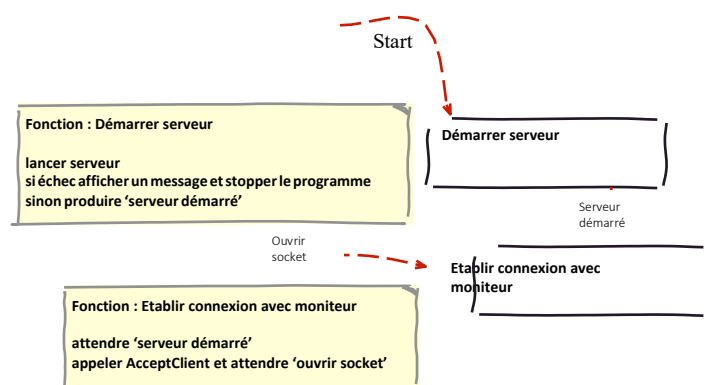


Figure 5. Intégration de la fonction découlant de l'exigence 2

...

Sur la Figure 6, on note l'introduction de deux données partagées entre plusieurs fonctions : mouvement et robot démarré. Quelle en est l'interprétation ? A la lecture du diagramme d'architecture fonctionnelle, le choix fait par le concepteur pour traiter les mouvements consiste à envoyer périodiquement un message de mouvement au robot. Pour cela, la fonction lit le mouvement à réaliser dans une variable appelée 'mouvement'. Cette variable est mise à jour lorsqu'un message de mouvement est reçu. Cependant, le message ne doit être envoyé que si le robot est démarré, d'où l'ajout de la variable robot démarré qui est mise à jour dans la fonction qui démarre le robot. On aurait pu imaginer une autre conception... A cette étape, on commence donc à faire, implicitement ou pas, des choix de modélisation (sur lesquels vous pourrez revenir par la suite).

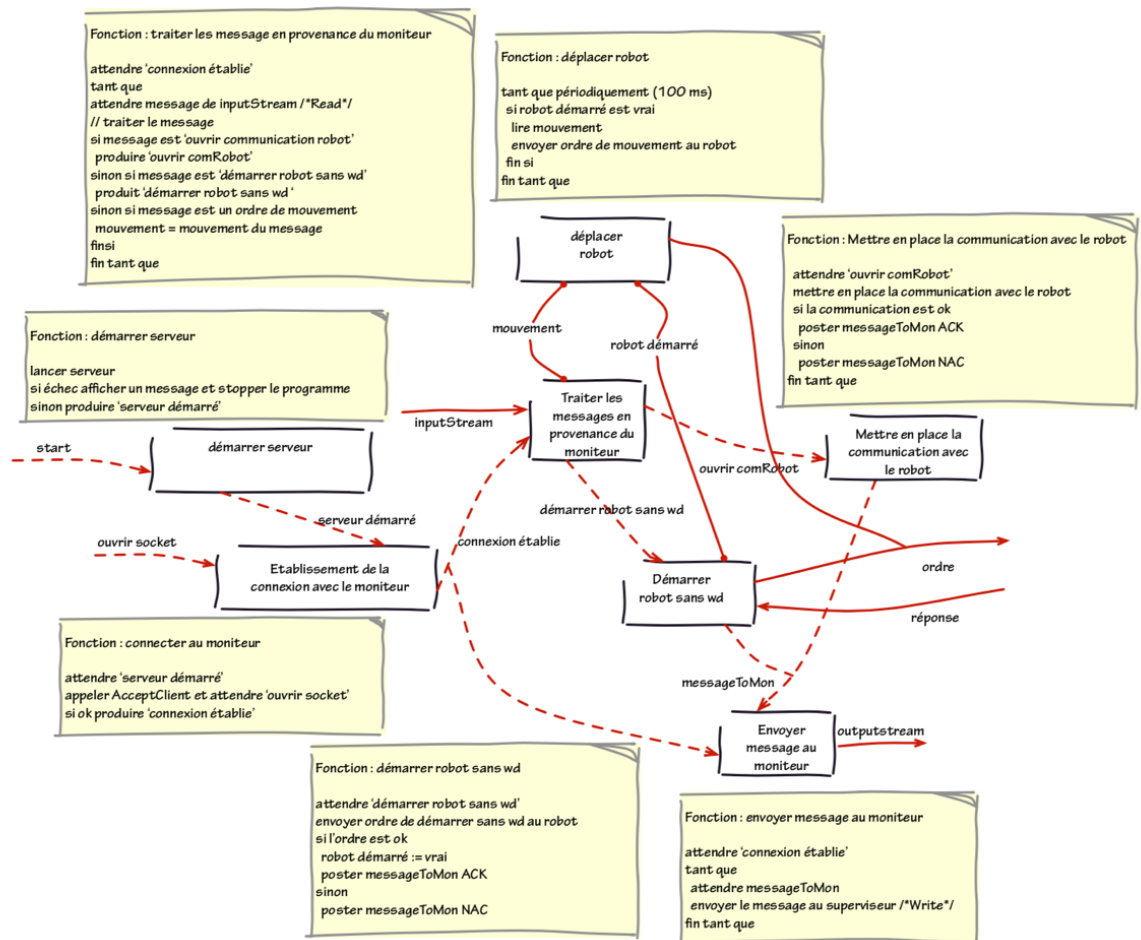


Figure 6. Étape intermédiaire du diagramme d'architecture fonctionnelle statique du superviseur

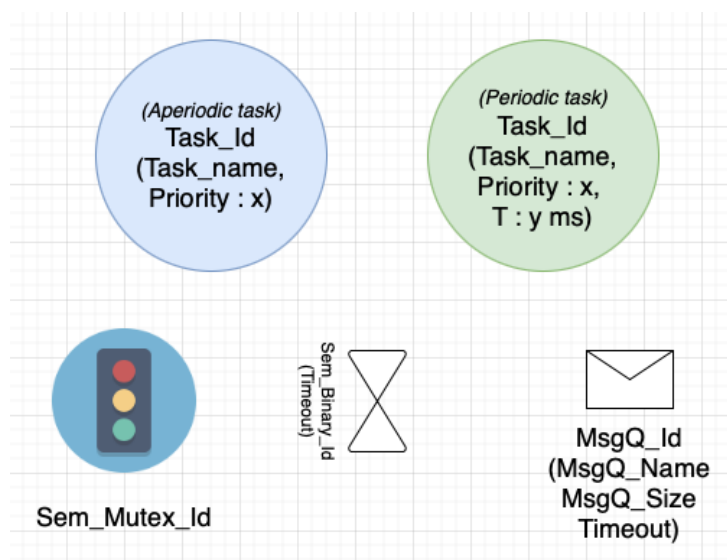
3.2 Conception du superviseur

L'étape de conception permet de déterminer comment le système va faire pour exécuter ses fonctions compte tenu des services offerts par la plateforme logicielle choisie (Xenomai).

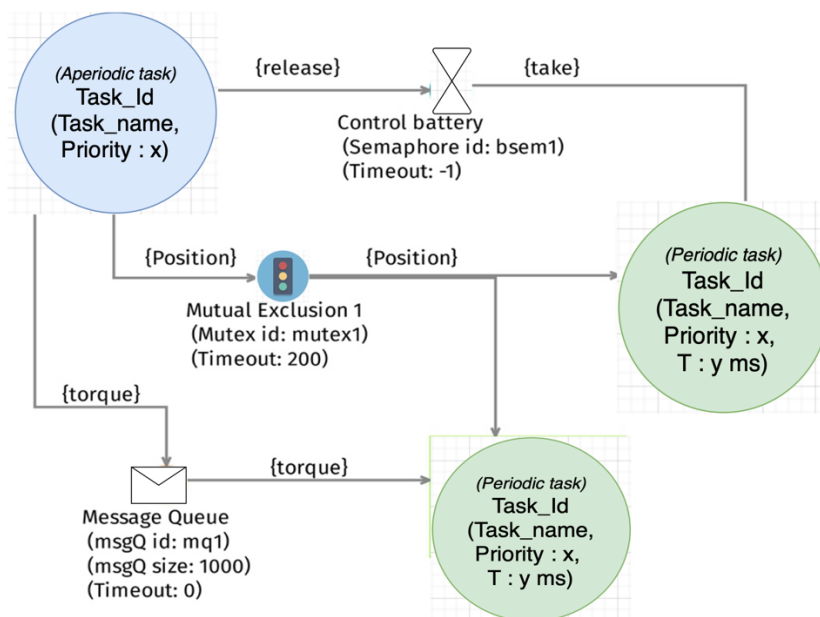
L'objectif est ici de déterminer l'architecture physique de l'application et d'en faire le diagramme. Comme pour le diagramme d'architecture fonctionnelle, la vue statique fait apparaître les composants logiciels appelés tâches (ou threads), les échanges de données et les synchronisations

entre les tâches, ainsi que les moyens choisis (services de Xenomai) pour les implanter. La vue dynamique représente l'exécution des tâches dans le temps.

NB : Vous pouvez, pour représenter l'architecture physique statique, utiliser le formalisme de la figure ci-dessous. (Voir sous Moodle le fichier « Architecture physique.drawio » (édité sous <https://app.diagrams.net/>).



Éléments de représentation sous Draw.io



Exemple de représentation d'un diagramme d'architecture physique

Sur le plan de la démarche, vous devez pour cela tout d'abord réfléchir à un découpage en tâches qui vous semble pertinent. Vous effectuerez un découpage 'optimal' de l'application en tâches, que vous justifierez ; ces tâches vont exécuter les différentes fonctions en intégrant et respectant des contraintes de parallélisme, de séquençement dans le temps, de temps, de partage de ressources, ... (on peut donc être amené, selon le niveau de détail auquel vous avez fait l'analyse fonctionnelle, à faire exécuter une fonction par plusieurs tâches ou au contraire à regrouper plusieurs fonctions dans une même tâche).

Vous devez ensuite caractériser les tâches (nom, priorité¹, période, entrées, sorties), leurs activations (périodique, sur événement...).

Puis choisir, caractériser et justifier des moyens de communication (variable globale protégée ou pas par un mutex, file de messages (taille, format des messages, timeouts...) et de synchronisation (sémaphore binaire, suspension/réveil, ...).

NB : Assurez-vous que vous retrouvez au niveau des tâches les mêmes flux d'entrées/sorties (données et contrôles) que pour les fonctions exécutées par ces tâches (cohérence avec le diagramme d'architecture fonctionnelle et, au-delà, avec les exigences).

Et enfin décrire l'activité globale de l'application logicielle (entrelacement des flux d'exécutions des différentes tâches dans le temps) ainsi que l'activité interne des tâches (par des diagrammes en SysML : le comportement de chaque tâche pourra par exemple être décrit à l'aide d'un diagramme d'activité).

Pour finir, préparez le développement et les tests (tests unitaires et tests d'intégration) : quelle tâche allez-vous commencer par coder ? Comment allez-vous la tester ? Quelle autre tâche allez-vous coder ensuite ? Comment allez-vous la tester ? Comment allez-vous tester que les deux tâches fonctionnent ensemble comme vous l'avez spécifié ? Comment vérifier que les contraintes de temps sont bien respectées ? Etc.

A FAIRE durant les séances de TD (à terminer avant les TP)

Faire la spécification (1^{ère} séance) puis la conception (2^{ème} séance) du superviseur. Aidez-vous de la trame de rapport pour vous guider.

A FAIRE AVANT le premier TP :

Vous constaterez en analysant le code fourni que vous êtes contraints par la plateforme et que certains éléments logiciels sont déjà codés, prêt à être utilisés.

Vous devrez donc, avant la première séance de TP, parcourir le code fourni pour savoir ce qu'il contient et être capable, au moment du codage d'aller rapidement retrouver l'information.

4. TRAVAIL DE TP

4.1 Implantation (codage et tests) du superviseur

L'objectif des TP est de programmer et valider l'application logicielle dont vous avez défini l'architecture en TD, **dans sa version de base** (des questions complémentaires vous seront proposées en TP en fonction de votre avancement).

Pour cela, vous allez travailler votre projet en binôme, tout en vous répartissant le travail de codage et d'intégration (prenez bien soin à la définition des interfaces entre les tâches pour cela). Vous prévoirez ensemble les activités à mener et les mènerez effectivement selon votre planning. Le client (enseignant de TP) est présent pour valider de façon incrémentale votre travail quand vous le lui demandez.

¹ Pour déterminer les priorités, voici quelques règles : pour les tâches périodiques, on attribue les priorités par ordre décroissant des périodes, c'est-à-dire qu'une tâche sera d'autant plus prioritaire que sa période sera petite. Pour les tâches apériodiques, le niveau de priorité va dépendre de la criticité des fonctions.

Lors de la première séance, vous allez vous familiariser avec l'environnement de développement et le code fourni, vous répartir le travail, et déterminer la stratégie de développement/mise au point.

Décrire les fonctionnalités du code de base fourni

A la fin de cette séance, vous devez :

- savoir vous connecter à la cible et avoir compris en quoi consistent les opérations effectuées,
- avoir identifié le travail à faire sur les prochaines séances et planifié ce travail,
- avoir poursuivi la rédaction du rapport, initié durant les TD (en indiquant par exemple quelle est votre stratégie de code et intégration, et pourquoi)
- avoir fait valider votre diagramme d'architecture physique par votre enseignant de TP.

Lors des séances suivantes, vous vous organiserez pour atteindre vos objectifs, en veillant à revoir votre planning si nécessaire afin que vous soyez toujours en mesure de livrer au client à temps.

Nous vous proposons les étapes suivantes :

1. Démarrer le robot (sans watchdog) et afficher périodiquement l'état de la batterie
2. Contrôler le mouvement du robot avec les flèches de direction du moniteur
3. Ouvrir la caméra et afficher l'image sur le moniteur
4. Récupérer la position du robot et l'afficher sur le moniteur
5. Démarrer le robot avec watchdog et gérer la remise à zéro périodique du watchdog
6. Piloter le mouvement du robot de manière autonome avec la caméra et les infos de positions
7. Gère la perte de communication avec le robot

A la fin de chaque séance, vous ferez une livraison au client de votre état d'avancement et un point sur le travail restant à faire. Vous travaillerez également sur la rédaction du rapport de façon incrémentale.

NB : Pensez à **sauvegarder les versions du logiciel** (avec commentaires indiquant le niveau d'avancement), que vous figez à la fin de chaque version fonctionnelle (pensez à sauvegarder ces versions et à chaque nouveau développement d'une fonctionnalité de faire une copie sur laquelle vous allez développer).

Lors de la dernière séance se tiendra la livraison au client : vous lui ferez une présentation formelle du logiciel, démonstration des performances et du respect des exigences, argumentaire sur les choix de conception et réalisation ; vous lui livrerez à l'issue de la séance le code ainsi que le rapport.